

From Apparent KNOB Downgrade to HCI Reporting False Positive: Experimental Analysis on Raspberry Pi 5 / BCM4345C0

Marco Stocco, Riccardo Citron
Università degli studi di Padova

Abstract—The KNOB attack exploits the unauthenticated Bluetooth BR/EDR key-size negotiation to reduce the effective encryption entropy of a connection. This work investigates whether a complete KNOB-style downgrade can be reproduced on a Raspberry Pi 5 equipped with a BCM4345C0/CYW43455 Bluetooth controller. Initial experiments appeared to indicate a successful downgrade: modifying a byte at offset `+0xA7` in the controller connection structure caused both InternalBlue and the HCI command `Read Encryption Key Size` to report a 1-byte key size. However, timing validation with `btmon` and vendor diagnostic traces showed that immediately after `Encryption Change` the controller still reported a 16-byte key size, while altered values such as 0 and 1 appeared only later after direct RAM writes. We therefore classify the observed downgrade as an HCI/InternalBlue reporting false positive rather than a real LMP-level entropy downgrade. We further searched for upstream RAM fields that could influence the key-size negotiation, but none of the tested candidates produced a natural 1-byte key-size result without modifying the reporting field. Our results show that, with the available Raspberry Pi 5 setup, local key-size reporting can be manipulated, but a complete KNOB replication requires additional capabilities such as LMP-level modification, firmware patching, or over-the-air BR/EDR ciphertext capture.

1. INTRODUCTION

Bluetooth BR/EDR is still widely used to provide wireless connectivity for audio devices, smartphones, laptops, embedded platforms, and legacy peripherals. Although modern Bluetooth deployments include several layers of pairing, authentication, and encryption, the security of a BR/EDR encrypted link ultimately depends on parameters negotiated at the controller level. One of these parameters is the effective encryption key size, which determines the entropy of the link-layer encryption key used after the authentication phase.

The KNOB attack showed that the Bluetooth BR/EDR encryption key-size negotiation can be manipulated so that two devices agree on a very low-entropy encryption key, potentially as small as one byte [1]. In the original attack model, the adversary interferes with the Link Manager Protocol (LMP) negotiation before the final encryption key is derived and used. If the negotiated key size is reduced to one byte, the resulting key space becomes small enough to be exhaustively searched. A complete replication of KNOB must therefore demonstrate that the negotiated value used by the controllers during LMP encryption setup is actually reduced, not merely that a host-side diagnostic interface reports a smaller key size.

In this work, we investigate whether a complete KNOB-style key-size downgrade can be reproduced using a Raspberry Pi 5 equipped with its onboard Broadcom/Cypress BCM4345C0/CYW43455 Bluetooth controller. Our initial experiments appeared to indicate a successful downgrade: by modifying a byte in the controller RAM connection structure, InternalBlue reported an effective key length of one byte, and the standard HCI command `Read Encryption Key Size` returned the same value. This initially suggested that the Raspberry Pi 5 controller could be used to force a KNOB-like downgrade against a Bluetooth BR/EDR target.

However, a deeper validation revealed that this interpretation was incorrect. We found that the RAM field responsible for the apparent downgrade, located at offset `+0xA7` inside the active connection structure, affects the value reported to InternalBlue and to the local HCI interface, but does not by itself prove that the LMP-negotiated entropy was reduced. In particular, vendor diagnostic traces showed that immediately after the `Encryption Change` event, the controller still reported a key size of 16 bytes. Altered values such as 0 or 1 appeared only later, after direct RAM writes to the connection-structure field. This indicates that the observed downgrade is a post-negotiation reporting artifact rather than evidence of a real cryptographic downgrade.

This distinction is central to our study. A host-visible report of key size 1 is not equivalent to a link-layer negotiation of $N = 1$. The former can be affected by local controller state, while the latter requires modification or observation of the LMP key-size negotiation before key derivation. Since the Raspberry Pi 5 setup used in this work does not provide reliable access to LMP packet modification, ROM/patchram hooks, or over-the-air BR/EDR ciphertext capture, the original objective of reproducing a full KNOB attack could not be satisfied with the available equipment.

Instead of presenting the apparent downgrade as a successful attack, we reframe the project as an experimental analysis of the limitations of KNOB replication on the Raspberry Pi 5 / BCM4345C0 platform. This negative result is relevant because it identifies a realistic validation pitfall: relying only on InternalBlue or HCI-reported key sizes can lead to the false conclusion that a KNOB downgrade has occurred. Our experiments show that, on this platform, the reported key size can be forged through RAM manipulation without demonstrating that

the actual controller-level encryption entropy has changed.

The contributions of this work are as follows:

- We experimentally map relevant fields of the BCM4345C0 connection structure on Raspberry Pi 5, including the field at offset `+0xA7` used by InternalBlue and HCI to report the effective key length.
- We show that writing to this field can make InternalBlue and HCI Read Encryption Key Size report arbitrary or invalid key sizes, including 0 and 1 byte.
- We validate, through timing analysis of vendor diagnostic traces, that the first key-size reading immediately after encryption activation remains 16 bytes, while altered values appear only after post-encryption RAM writes.
- We test several candidate upstream RAM fields that could have acted as key-size inputs, and show that none of the tested candidates cause a natural downgrade to one byte without directly modifying the reporting field.
- We discuss the practical capabilities missing from the Raspberry Pi 5 setup for a complete KNOB replication, namely LMP-level visibility or modification, firmware patching before key derivation, and external BR/EDR ciphertext capture.

Overall, our results do not demonstrate a complete KNOB attack on the tested platform. Rather, they provide an experimentally grounded explanation of why the apparent downgrade observed through HCI/InternalBlue is insufficient, and they define the minimum additional capabilities required to move from HCI-level reporting manipulation to a real LMP-level entropy downgrade.

2. BACKGROUND

This section summarizes the Bluetooth BR/EDR mechanisms that are relevant to our experiments and clarifies the distinction between a real KNOB downgrade and an apparent downgrade observed only through host-side reporting interfaces.

2.1 Bluetooth BR/EDR, HCI, and LMP

Bluetooth BR/EDR separates the host stack from the controller. The host stack, running on the operating system, interacts with the controller through the Host Controller Interface (HCI). In contrast, low-level link-management operations are handled internally by the Bluetooth controller firmware. One of the most relevant controller-level protocols is the Link Manager Protocol (LMP), which controls authentication, encryption setup, and several BR/EDR link parameters [2].

This separation is central to our analysis. Tools such as `btmon`, `hcitool`, and Android `btsnoop` logs expose HCI-level activity between the host and the controller. They do not necessarily expose the LMP packets exchanged directly between Bluetooth controllers. Therefore, an HCI-level observation can be insufficient to prove what happened during the LMP-level negotiation.

2.2 BR/EDR Encryption Key Derivation

After pairing and authentication, Bluetooth BR/EDR establishes an encrypted link. The security specification defines how

link keys and encryption keys are generated and used to protect BR/EDR traffic [3]. Conceptually, the relevant chain for this work is:

$$K_L \rightarrow K_C \rightarrow K'_C, \quad (1)$$

where K_L is the link key derived from pairing, K_C is the encryption key generated during the encryption setup, and K'_C is the final reduced key used by the link-layer encryption algorithm. In the context of KNOB, the critical parameter is the effective encryption key size, denoted in this paper as N . A full-size BR/EDR encryption key uses up to 16 bytes of entropy, whereas a downgraded key may use a much smaller value.

2.3 The KNOB Attack

The KNOB attack exploits the fact that Bluetooth BR/EDR devices negotiate the encryption key size through unauthenticated LMP messages [1]. An active adversary can manipulate this negotiation and force both devices to agree on a low-entropy encryption key. In the most severe case, the negotiated entropy can be reduced to 1 byte, leaving only 256 possible keys.

The essential point is that the downgrade must occur before the final encryption key is derived and before encrypted traffic is protected with that key. In simplified form, the KNOB-relevant flow is:

$$\text{LMP negotiation} \rightarrow N \rightarrow E_s(K_C, N) \rightarrow K'_C. \quad (2)$$

Figure 1 summarizes this distinction. A complete replication of KNOB must affect the LMP key-size negotiation before K'_C is derived. Conversely, a value reported later through HCI or debugging tools is not, by itself, sufficient evidence that the negotiated entropy was actually reduced.

2.4 Why HCI-Level Evidence Is Not Sufficient

HCI provides commands and events that allow the host to interact with the Bluetooth controller and inspect selected controller states. For example, the command `HCI Read Encryption Key Size` returns the encryption key size associated with a connection handle. However, this command reports a controller-maintained value after the encryption procedure has taken place. It does not expose the original LMP packets, nor does it prove that the reported value was the one used during key derivation.

This distinction becomes especially important when the local controller memory can be modified. If an attacker can write to controller RAM, then a host-visible field used for reporting may be changed after encryption has already been established. In that case, the host may observe:

$$\text{HCI Read Encryption Key Size} = 1$$

even though the actual LMP negotiation may have completed with:

$$N = 16.$$

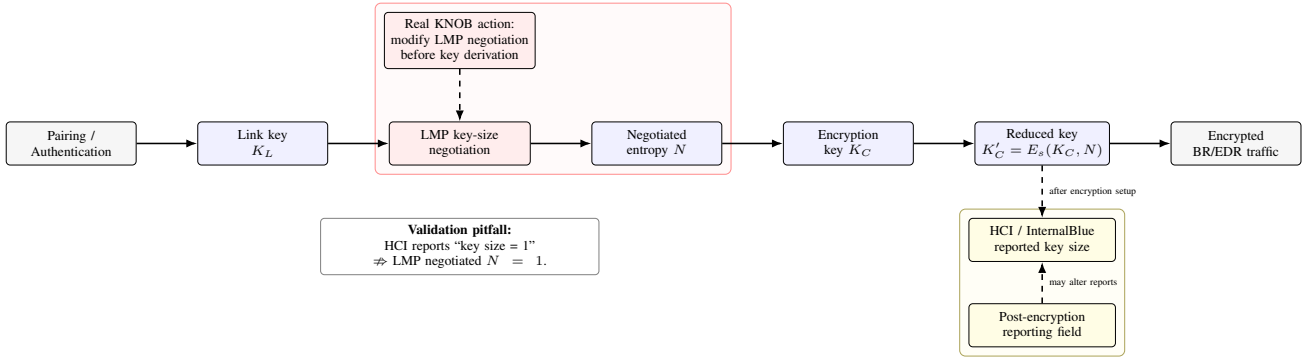


Fig. 1: KNOB-relevant BR/EDR encryption flow. A real KNOB downgrade must affect the LMP key-size negotiation before the reduced encryption key K'_C is derived. In contrast, modifying a post-encryption reporting field can change the key size reported through HCI or InternalBlue without proving that the negotiated entropy N was actually reduced.

A rigorous KNOB validation must therefore distinguish between:

- the *reported* key size visible at HCI or debugging level;
- the *negotiated* key size exchanged through LMP;
- the *effective cryptographic entropy* of the key actually used to protect the BR/EDR link.

Our experimental results show that this distinction is not only theoretical. On the tested Raspberry Pi 5 / BCM4345C0 platform, a controller RAM field can be modified so that InternalBlue and HCI Read Encryption Key Size report a key size of 1 byte, even though the first key-size reading immediately after Encryption Change remains 16 bytes. This observation motivated the false-positive analysis described in the following sections.

2.5 Implications for Experimental Replication

The original KNOB attack was implemented by acting at a lower level than HCI reporting. A complete replication requires either visibility into LMP negotiation, the ability to modify LMP packets before encryption setup, or an equivalent cryptographic proof based on over-the-air ciphertext. In contrast, our Raspberry Pi 5 setup provides partial controller RAM access but does not provide a reliable way to patch the LMP negotiation path, dump the relevant ROM code, or capture BR/EDR ciphertext over the air.

Consequently, our experimental question is not only whether a key size of 1 can be displayed locally, but whether such a value corresponds to a real controller-level entropy downgrade. The remainder of the paper shows that, with the current setup, the apparent downgrade is better explained as an HCI/InternalBlue reporting artifact rather than a complete KNOB replication.

3. SYSTEM MODEL AND THREAT MODEL

This section defines the entities, attacker capabilities, and experimental limitations considered in our study. The goal is to make explicit which parts of the Bluetooth stack are observable or controllable in our setup, and which parts remain outside our reach.

3.1 Entities

We consider two Bluetooth BR/EDR devices connected through a standard encrypted link. The first device is the attacker-controlled platform, while the second device is the target.

a) Attacker-controlled device.

The attacker-controlled device is a Raspberry Pi 5 equipped with its onboard Broadcom/Cypress BCM4345C0/CYW43455 Bluetooth controller. The attacker has local administrative access to the Raspberry Pi and can use InternalBlue to inspect and modify parts of the local controller RAM. The Raspberry Pi acts as a legitimate Bluetooth peer in the connection, rather than as an external over-the-air sniffer.

b) Target device.

The primary target device is a Samsung Galaxy Ace Style 2014, identified by the Bluetooth address $F8:84:F2:62:96:AA$. The target participates in a standard Bluetooth BR/EDR connection with the Raspberry Pi. During the experiments, Android Bluetooth HCI snoop logging was enabled on the target in order to obtain an independent HCI-level trace of the connection.

c) Host-side analysis platform.

A Windows host was used to extract and analyze the Samsung HCI snoop log through ADB, Wireshark, and `tshark`. This host was not part of the Bluetooth link itself; it was used only for offline analysis of logs and experimental artifacts.

3.2 Attacker Capabilities

The attacker has the following capabilities on the Raspberry Pi side:

- establish, disconnect, and reconnect BR/EDR links with the target device using the Linux Bluetooth stack;
- inspect local controller state through InternalBlue, including active connection metadata;
- read selected controller RAM regions through InternalBlue;
- write selected controller RAM locations through InternalBlue;

- query local HCI-visible state, including the value returned by HCI Read Encryption Key Size;
- capture HCI-level traces using `btmon`;
- generate L2CAP traffic using `l2ping`;
- extract and analyze the target-side Android HCI snooper log.

These capabilities are sufficient to observe and manipulate some local controller state after a connection has been established. In particular, they allow us to determine whether a local controller RAM modification changes the value reported through InternalBlue or HCI.

3.3 Attacker Limitations

The attacker does not have the following capabilities in the considered setup:

- direct over-the-air capture of Bluetooth BR/EDR ciphertext;
- reliable decoding of LMP key-size negotiation packets exchanged between the controllers;
- a working LMP packet modification hook before encryption setup;
- reliable ROM dumping of the BCM4345C0 firmware;
- reliable patchram access to modify the firmware path responsible for LMP key-size negotiation;
- access to the target controller firmware or target controller RAM.

These limitations are central to the interpretation of the results. In particular, the attacker can modify host-visible state inside the local controller, but cannot directly prove from this capability alone that the remote controller negotiated the same downgraded entropy value at the LMP level.

3.4 Threat Model

Our initial objective was to evaluate whether the Raspberry Pi 5 could be used to reproduce a KNOB-style entropy downgrade against the target device. In a complete KNOB attack, the adversary must influence the Bluetooth BR/EDR encryption key-size negotiation so that the devices derive and use a low-entropy encryption key. This requires acting before or during the LMP encryption setup.

In contrast, our experimental attacker is not assumed to be a fully capable over-the-air man-in-the-middle. The attacker controls one endpoint of the connection, namely the Raspberry Pi 5, and can inspect and manipulate only its own controller state. This model allows us to test whether the controlled endpoint can be forced into a low reported key-size state, but it does not automatically provide evidence that the LMP negotiation itself was modified.

Figure 2 summarizes the system and threat model. The Raspberry Pi exposes HCI-level traces and selected writable RAM fields. The Samsung target exposes only host-side HCI logs. The LMP negotiation and the over-the-air encrypted BR/EDR traffic remain below the level directly observable by our setup.

3.5 Security Objective

The original security objective was to determine whether the attacker-controlled Raspberry Pi could force the connection into a real 1-byte encryption entropy state. A successful outcome would require evidence that the value N used during LMP key-size negotiation was actually reduced to 1 before the final encryption key was derived.

However, given the capabilities and limitations above, we refine the experimental objective as follows. We first test whether the local controller can be made to report a 1-byte key size. We then evaluate whether this reported downgrade corresponds to a real protocol-level downgrade or to a post-negotiation reporting artifact.

This refinement is necessary because the attacker can modify local RAM fields that affect HCI-visible state. Therefore, the relevant research question is not simply whether HCI Read Encryption Key Size can be made to return 1, but whether such a value can be linked to the actual LMP-negotiated entropy.

3.6 Adversarial Interpretation

Throughout the remainder of this paper, we distinguish between three increasingly strong claims:

- 1) **Reported downgrade:** InternalBlue or HCI reports a key size of 1 byte.
- 2) **Negotiated downgrade:** LMP key-size negotiation actually results in $N = 1$.
- 3) **Cryptographic downgrade:** the BR/EDR traffic is protected with a key whose effective entropy is 1 byte.

Our setup can directly test the first claim. It can partially investigate the second claim through timing analysis and indirect traces, but it cannot directly observe or modify LMP negotiation. It cannot establish the third claim without an external BR/EDR ciphertext capture or equivalent cryptographic evidence.

This hierarchy of claims is used in the following sections to interpret the apparent downgrade produced by modifying the controller RAM field at offset `+0xA7`.

4. EXPERIMENTAL SETUP

This section describes the experimental environment used to evaluate the feasibility of reproducing a KNOB-style downgrade on the Raspberry Pi 5 / BCM4345C0 platform. While the previous section defined the system and threat model, this section focuses on the concrete hardware, software tools, logging procedures, and experimental artifacts used during the analysis.

4.1 Hardware Configuration

The attacker-controlled platform was a Raspberry Pi 5 using its onboard Broadcom/Cypress Bluetooth controller from the BCM4345C0/CYW43455 family. This controller was accessed through the Linux Bluetooth stack and through InternalBlue, which allowed partial inspection and modification of controller RAM.

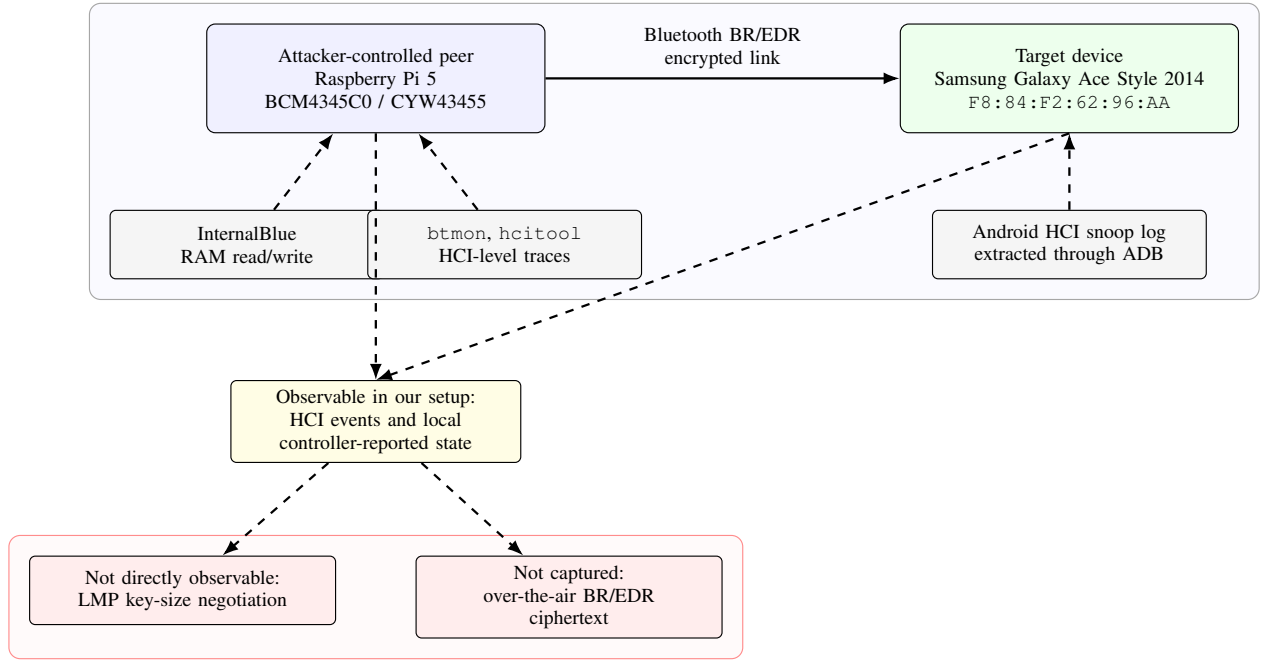


Fig. 2: System and threat model. The Raspberry Pi 5 is the attacker-controlled Bluetooth peer and exposes local HCI traces and selected controller RAM through InternalBlue. The Samsung device is the target and provides only host-side HCI snoop logs. The setup does not directly expose the LMP key-size negotiation or the over-the-air BR/EDR ciphertext, which are required to prove a complete KNOB downgrade.

The primary target device was a Samsung Galaxy Ace Style 2014 with Bluetooth address:

F8:84:F2:62:96:AA.

The Samsung device was used as the main BR/EDR target because it established stable Bluetooth connections with the Raspberry Pi and used E0 encryption in the tested configuration. This made it suitable for studying KNOB-relevant behavior on a legacy BR/EDR target.

A Windows machine was used as a secondary analysis platform. It was not part of the Bluetooth link, but it was used to extract Android HCI snoop logs through ADB and to analyze them with Wireshark and tshark.

4.2 Software Tools

Table I summarizes the main tools used during the experiments.

On the Raspberry Pi, `bluetoothctl` was used to manage the BR/EDR connection with the Samsung target. `btmon` was used to record controller events, including Encryption Change and Read Encryption Key Size. `hcitool` was used to issue explicit HCI commands to the local controller, while `l2ping` was used to generate traffic after the link had been established.

InternalBlue was used for controller-side inspection. In particular, the command `info connections` was used to inspect active connection metadata, and `hexdump` and `writemem` were used to inspect and modify selected controller RAM locations.

4.3 Connection Establishment

The Samsung target was paired and connected to the Raspberry Pi through the standard Linux Bluetooth stack. During the main experiments, the Raspberry Pi acted as the local controlled peer, while the Samsung acted as the remote target.

A typical connection setup used the following operations:

```
bluetoothctl
power on
agent NoInputNoOutput
default-agent
connect F8:84:F2:62:96:AA
info F8:84:F2:62:96:AA
```

After the connection was established, InternalBlue was used to identify the active connection entry in the controller memory. The connection handle reported by InternalBlue was then used to query the encryption key size through HCI.

The HCI command used for this purpose was:

```
hcitool cmd 0x05 0x0008 <handle_lsb> <handle_msb>
```

which corresponds to HCI Read Encryption Key Size. In a normal encrypted connection, this command returned a key size of `0x10`, corresponding to 16 bytes.

4.4 Controller RAM Inspection

A central part of the analysis consisted of inspecting the active connection structure inside the BCM4345C0 controller RAM. Through repeated experiments, we identified a connection-array layout in which active connection entries are separated by a fixed stride. The key-size field that initially produced the

TABLE I: Experimental setup and tools used throughout the analysis.

Component	Tool / Device	Purpose
Attacker-controlled platform	Raspberry Pi 5	Bluetooth peer and controller experimentation platform
Bluetooth controller	BCM4345C0 / CYW43455 family	Local controller under analysis
Firmware framework	InternalBlue	RAM inspection, connection metadata, memory writes
Target device	Samsung Galaxy Ace Style 2014	Main BR/EDR target device
Target BD address	F8:84:F2:62:96:AA	Stable identifier for connection experiments
Linux HCI tracing	btmon	HCI and vendor diagnostic traces
Linux HCI commands	hcitool	HCI Read Encryption Key Size queries
Connection management	bluetoothctl	Pairing, trusting, connecting, disconnecting
Traffic generation	l2ping	L2CAP Echo traffic over the encrypted link
Android log extraction	ADB	Samsung HCI snoop log extraction
Packet analysis	Wireshark, tshark	Offline inspection of HCI and btspoof traces

apparent downgrade was located at offset `+0xA7` from the base of the active connection structure.

The address of this field was computed as:

```
key_len_addr = connection_array_base
+ slot_index * connection_struct_size
+ 0xA7. (3)
```

For the stable Samsung experiments, this field was observed at:

0x20557F.

Writing `0x01` to this address made InternalBlue report an effective key length of 1 byte and made HCI Read Encryption Key Size return 1 on the Raspberry Pi side. This behavior motivated the initial hypothesis that the field could control the negotiated entropy. Later validation, described in Section 7, showed that this interpretation was incorrect and that the field behaves as a post-negotiation reporting field.

4.5 Raspberry Pi-Side Logging

The main Raspberry Pi-side traces were collected with `btmon`. These traces were used to observe connection events, encryption activation, HCI commands, and the timing of key-size reads. In particular, we used the relative timing between Encryption Change and Read Encryption Key Size to determine whether an observed key-size value appeared before or after encryption had already been enabled.

A typical trace of interest contained the sequence:

```
Encryption Change
Read Encryption Key Size
Key size: 16
```

This baseline sequence was later compared against traces collected after RAM modifications. The decisive validation trace showed that the first key-size read immediately after encryption still returned 16 bytes, while altered values appeared only later after direct memory writes.

4.6 Target-Side HCI Snoop Logging

To obtain an independent target-side view, Bluetooth HCI snoop logging was enabled on the Samsung device

through Android Developer Options. The expected path `/sdcard/btsnoop_hci.log` was not present on this device. Instead, the valid log was found at:

`/sdcard/Android/data/btsnoop_hci.log`.

The log was extracted using ADB:

```
adb pull /sdcard/Android/data/btsnoop_hci.log \
samsung_btsnoop_plusA7_today.log
```

The extracted file was analyzed with `tshark`. The Samsung-side timeline confirmed that the target observed the connection and encryption setup. The relevant sequence included:

```
Connect Complete
Link Key Request
Link Key Request Reply
Encryption Change
Disconnect Complete
```

However, the Samsung-side log did not contain a target-side HCI Read Encryption Key Size command. Therefore, the Samsung HCI snoop log confirmed that encryption was enabled but did not reveal the key size as perceived by the target controller.

4.7 Traffic Generation

After the encrypted link was established, `l2ping` was used to generate real L2CAP traffic between the Raspberry Pi and the Samsung target:

```
l2ping -i hci0 -s 600 -c 30 F8:84:F2:62:96:AA
```

This step was important because it allowed us to verify whether the connection remained usable after local controller RAM modifications. If a RAM write had changed the active cryptographic key only on the Raspberry Pi side, the two controllers would no longer share the same encryption state and L2CAP traffic would be expected to fail. In the observed experiments, traffic could continue even after modifying the local reported key-size field, which supported the interpretation that the modification affected reporting rather than the live cryptographic key.

4.8 Collected Artifacts

The experimental analysis relied on the following artifacts:

- InternalBlue `info connections` outputs before and after RAM modifications;
- controller memory hexdumps around connection structures and candidate fields;
- `btmon` traces collected on the Raspberry Pi;
- HCI command outputs from `hcitool`;
- Samsung-side `btsnoop` logs extracted through ADB;
- `tshark` timelines generated from the Samsung logs;
- `l2ping` logs showing whether L2CAP traffic continued after RAM writes.

Together, these artifacts allowed us to compare three different views of the experiment: the controller state reported by InternalBlue, the HCI-visible state reported by the Raspberry Pi controller, and the target-side HCI timeline observed on the Samsung device. The following sections use these artifacts to reconstruct the apparent downgrade and to validate whether it corresponds to a real KNOB entropy reduction.

5. METHODOLOGY

This section describes the experimental methodology used to move from an apparent key-size downgrade to a stricter validation of whether a real KNOB downgrade had occurred. The methodology was designed around one central requirement: distinguishing a host-visible reported key size from the entropy value actually negotiated at the controller level.

5.1 Methodological Overview

Our analysis followed the workflow shown in Figure 3. We first established a standard BR/EDR connection between the Raspberry Pi and the Samsung target. We then located the active connection structure in the local controller RAM and identified the field that controls the key size reported by InternalBlue and HCI. After observing an apparent downgrade, we validated it through timing analysis, target-side HCI logging, traffic continuity tests, and a search for possible upstream RAM variables.

5.2 Connection-Structure Mapping

The first step was to identify the active connection structure used by the BCM4345C0 controller. InternalBlue’s `info connections` command was used to obtain the active connection handle, remote Bluetooth address, role information, reported key length, and link-key-related metadata. We then inspected the corresponding RAM regions through targeted hexdump commands.

The active connection entry was identified by correlating InternalBlue metadata with values observed in RAM, including the remote device address and connection-specific cryptographic material. Previous sessions also verified several offsets in the connection structure by comparing RAM values against independently computed authentication outputs. This provided confidence that the inspected structure was the active controller-side representation of the Bluetooth connection.

For the Samsung target, the field later associated with the reported key size was found at offset `+0xA7` from the active

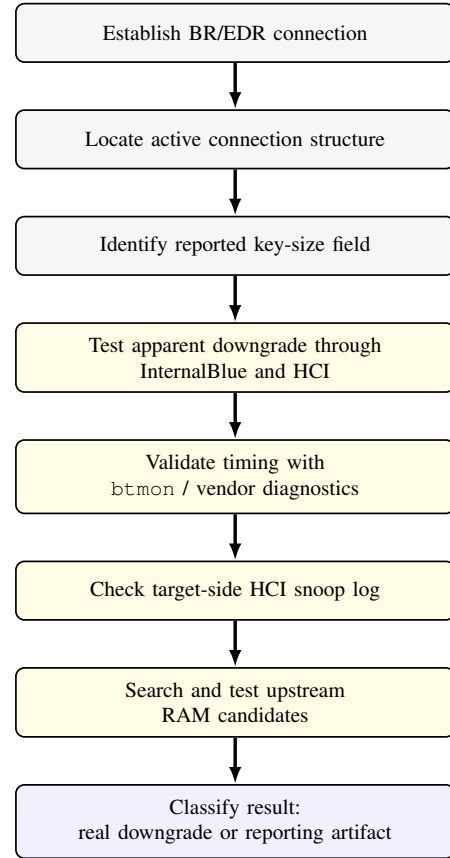


Fig. 3: Experimental methodology. The analysis starts from a standard BR/EDR connection, identifies the controller RAM field responsible for the apparent key-size downgrade, and then validates whether the observation corresponds to a real LMP-level downgrade or only to a post-negotiation reporting artifact.

connection-structure base. This offset was then reused as the reference point for the apparent downgrade experiments.

5.3 Apparent Downgrade Procedure

After establishing an encrypted BR/EDR connection, we modified the byte at offset `+0xA7` in the active connection structure. The write operation was performed through InternalBlue:

```
writemem <key_len_addr> 01 --hex
```

The immediate effect was checked in two ways. First, we used InternalBlue `info connections` to inspect the reported effective key length. Second, we queried the local controller using:

```
hcitool cmd 0x05 0x0008 <handle_lsb> <handle_msb>
```

which corresponds to HCI Read Encryption Key Size. When `+0xA7` was set to `0x01`, both InternalBlue and HCI reported a key size of 1 byte. This was the initial apparent downgrade.

However, this observation alone was not considered sufficient evidence of a real KNOB attack. Since the write was

performed on local controller RAM, the reported value could correspond to a mutable state field rather than to the value negotiated through LMP. The remaining methodology was therefore designed to validate or reject this interpretation.

5.4 Timing-Based Validation

To determine whether the reported key-size change occurred before or after encryption activation, we collected Raspberry Pi-side traces with `btmon` and vendor diagnostic events. The key events of interest were:

```
Encryption Change
Read Encryption Key Size
Key size: <value>
```

The validation criterion was the following. If a real downgrade had occurred during encryption setup, the first key-size reading immediately after `Encryption Change` should already reflect the downgraded value. Conversely, if the first post-encryption reading remained 16 bytes and lower values appeared only after explicit RAM writes, the observation would support the reporting-artifact hypothesis.

This timing test became the decisive validation step. It allowed us to distinguish between a value used during key derivation and a value modified after the link was already encrypted.

5.5 Traffic Continuity Check

We also used L2CAP traffic continuity as an additional consistency check. After changing the reported key-size field, we generated traffic using:

```
l2ping -i hci0 -s 600 -c 30 F8:84:F2:62:96:AA
```

If the RAM write had changed the active cryptographic key only on the Raspberry Pi side, the two controllers would no longer share the same encryption state, and encrypted L2CAP traffic would be expected to fail. The fact that traffic continued after post-encryption modifications suggested that the active link key had not been changed by the write. This check is not, by itself, a cryptographic proof, but it is consistent with the interpretation that `+0xA7` affects reporting rather than live encryption.

5.6 Target-Side HCI Verification

To obtain an independent target-side view, we extracted the Samsung Bluetooth HCI snoop log and analyzed it with `tshark`. The purpose of this step was to check whether the target host issued `HCI Read Encryption Key Size` or otherwise exposed the key size perceived by the Samsung controller.

The Samsung trace confirmed connection establishment, link-key handling, and encryption activation. However, it did not contain a target-side `HCI Read Encryption Key Size` command. Therefore, the target-side log was useful to confirm that the connection and encryption setup occurred, but it was inconclusive with respect to the actual negotiated entropy value.

5.7 Search for Upstream Key-Size Inputs

After classifying `+0xA7` as a candidate reporting field, we searched for possible upstream RAM variables that could influence the actual LMP key-size negotiation. The search was based on two complementary strategies.

First, we compared RAM snapshots before and after connection establishment to identify dynamic bytes that changed to `0x10`, the normal 16-byte key-size value. Second, we searched for stable aligned values equal to `0x10`, since an upstream default key-size parameter might remain constant across pre- and post-connection snapshots.

Candidate addresses were filtered manually to discard obvious false positives, including pointers, SDP/L2CAP buffers, code-like regions, and structural fields whose modification broke the connection. Each remaining candidate was tested with the same criterion:

- 1) do not modify `+0xA7`;
- 2) write the candidate value to `0x01` before or during a controlled connection attempt;
- 3) establish the BR/EDR connection;
- 4) verify whether InternalBlue and HCI naturally report a key size of 1 byte.

Only a candidate that produced a key size of 1 without directly modifying the reporting field would be considered evidence of a possible upstream input. The results of this candidate search are reported in Section 8.

5.8 Classification Criteria

At the end of each experiment, the observation was classified according to the following criteria:

- **Reporting-only effect:** InternalBlue or HCI reports a modified key size only after direct writes to a reporting field, especially after encryption has already been enabled.
- **Potential upstream effect:** a candidate field causes the reported key size to become 1 without directly modifying `+0xA7`.
- **Real KNOB evidence:** LMP negotiation is observed or modified so that $N = 1$, or encrypted BR/EDR traffic is shown to be protected by a 1-byte entropy key.

Using these criteria, the apparent downgrade produced by `+0xA7` was not treated as a real KNOB replication unless it could be linked to a pre-encryption LMP-level change. This conservative classification is essential for avoiding false positives in Bluetooth firmware experiments.

6. INITIAL APPARENT DOWNGRADE

This section describes the initial observation that appeared to indicate a successful KNOB-style downgrade on the Raspberry Pi 5 / BCM4345C0 platform. The purpose is not to claim a real attack at this stage, but to explain why the result initially looked compatible with a key-size reduction.

6.1 Identification of the Reported Key-Size Field

After establishing a BR/EDR connection with the Samsung Galaxy Ace Style 2014, we inspected the active connection structure through InternalBlue. The field later associated with the reported key size was found at offset +0xA7 from the active connection-structure base.

For the stable Samsung connection slot, this produced the address:

0x20557F.

Before modification, InternalBlue reported the normal effective key length:

```
Effective Key Len: 16 byte (128 bit)
```

This matched the expected full-size BR/EDR encryption key size.

6.2 Memory Write Experiment

We then modified the byte at the identified address using InternalBlue:

```
writemem 0x20557F 01 --hex
```

After the write, InternalBlue reported:

```
Effective Key Len: 1 byte (8 bit)
```

At this point, the local controller state appeared to reflect a 1-byte key size. Since the KNOB attack aims to reduce the effective encryption entropy to 1 byte, this observation initially seemed consistent with a successful downgrade.

6.3 HCI-Level Confirmation

To check whether the same value was also visible through the standard HCI interface, we queried the local controller with HCI Read Encryption Key Size:

```
hctool cmd 0x05 0x0008 0B 00
```

In the corresponding experiment, the command returned a final byte equal to 0x01, indicating that the local HCI interface also reported a key size of 1 byte.

This agreement between InternalBlue and HCI made the result appear stronger than an InternalBlue-only artifact. The observation could be summarized as follows:

Observation point	Reported key size
InternalBlue info connections	1 byte
HCI Read Encryption Key Size	1 byte

6.4 Initial Interpretation

The initial interpretation was that modifying +0xA7 might influence the effective encryption key length used by the controller. Under this interpretation, the observed behavior would suggest that the Raspberry Pi controller had entered a low-entropy state compatible with the KNOB attack objective.

However, this interpretation required further validation. In particular, the experiment had to determine whether the value

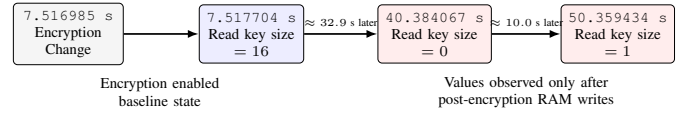


Fig. 4: Timing evidence for the false-positive validation. The first key-size reading immediately after Encryption Change reports 16 bytes. Altered values such as 0 and 1 appear only later, after direct RAM writes to the local reporting field.

reported by InternalBlue and HCI was actually the value negotiated through LMP before encryption setup, or whether it was only a mutable controller state field used for reporting.

For this reason, we classify this stage as an *apparent downgrade*. The following section shows that the observed value of 1 byte was not sufficient evidence of a real KNOB downgrade.

7. FALSE POSITIVE VALIDATION

This section validates the apparent downgrade described in Section 6. The objective was to determine whether the reported key size of 1 byte reflected a real LMP-level entropy downgrade, or whether it was only a post-negotiation reporting effect caused by modifying local controller RAM.

7.1 Validation Goal

A real KNOB downgrade must affect the key-size negotiation before the final encryption key is derived and before the link becomes encrypted. Therefore, the key validation question was:

What key size does the controller report immediately after encryption is enabled?

If the downgrade were real, the first HCI Read Encryption Key Size response immediately after Encryption Change should already report 1 byte. Conversely, if the controller reports 16 bytes immediately after encryption and only later reports 0 or 1 after direct RAM writes, then the modified field is a reporting artifact rather than evidence of a real KNOB downgrade.

7.2 Timing Evidence

The decisive trace was collected with btmon and vendor diagnostic events on the Raspberry Pi side. The relevant sequence is summarized in Figure 4.

The trace shows the following sequence:

7.516985	Encryption Change
7.517704	Read Encryption Key Size -> Key size: 16
40.384067	Read Encryption Key Size -> Key size: 0
50.359434	Read Encryption Key Size -> Key size: 1

The first key-size reading after encryption activation therefore reports 16 bytes, not 1 byte. The altered values appear approximately 32.9 and 42.8 seconds after Encryption

Change. These timings are incompatible with the interpretation that the values 0 and 1 were used during the original key-size negotiation.

7.3 Interpretation

The result shows that modifying the byte at offset `+0xA7` can affect the value returned by HCI Read Encryption Key Size, but this modification occurs after encryption has already been established. Consequently, the value reported by HCI after the write cannot be used as proof that the LMP negotiation selected $N = 1$.

The observation of a reported key size equal to 0 is especially important. A zero-byte encryption key size is not a valid KNOB outcome, yet the controller can be made to report it after a RAM write. This confirms that the HCI response is reading a mutable controller state field, rather than independently validating the cryptographic entropy of the active link.

Thus, the field at offset `+0xA7` is best interpreted as a post-negotiation reporting field. It controls the key size displayed through InternalBlue and returned through HCI, but it is not sufficient evidence of the value used during LMP key-size negotiation.

7.4 Traffic Continuity

After modifying the reported key-size field, L2CAP traffic could still be exchanged with the Samsung target. This behavior is consistent with the reporting-artifact interpretation. If the RAM write had changed the active encryption key only on the Raspberry Pi side, the two controllers would no longer share the same encryption state, and encrypted L2CAP traffic would be expected to fail.

This traffic-continuity check does not prove the exact negotiated entropy. However, it supports the conclusion that the post-encryption write did not replace the live link-layer encryption key with a different one.

7.5 Target-Side HCI Trace

The Samsung-side HCI snoop log provided an independent view of the connection timeline. The target trace showed connection establishment, link-key handling, and encryption activation:

```
Connect Complete
Link Key Request
Link Key Request Reply
Encryption Change
Disconnect Complete
```

However, the Samsung log did not contain a target-side HCI Read Encryption Key Size command. Therefore, it confirmed that the target observed the encrypted connection, but it did not expose the key size as perceived by the Samsung controller.

7.6 Conclusion

The validation disproves the initial interpretation of the apparent downgrade. Although InternalBlue and HCI can both be

made to report a key size of 1 byte, the timing evidence shows that the first post-encryption reading remains 16 bytes. Values 0 and 1 appear only after direct RAM writes performed after encryption is active.

We therefore classify the `+0xA7` method as an HCI/InternalBlue reporting false positive, not as a complete KNOB replication. A real KNOB downgrade would require evidence that the LMP-negotiated value N was reduced before encryption setup, or equivalent cryptographic evidence from the encrypted BR/EDR traffic.

8. SEARCH FOR UPSTREAM KEY-SIZE INPUTS

After validating that the field at offset `+0xA7` behaves as a post-negotiation reporting field, we searched for a possible upstream memory location that could influence the key-size value before or during the LMP negotiation. The goal was to find a field that, when set to `0x01`, would cause the controller to naturally report a 1-byte key size without directly modifying `+0xA7`.

8.1 RAM Diff Strategy

The first search strategy was based on pre- and post-connection RAM comparisons. We collected controller RAM snapshots before connecting the Samsung target and after the BR/EDR connection had been established. We then searched for bytes that changed to `0x10`, since `0x10` corresponds to the normal 16-byte encryption key size reported after a standard connection.

This dynamic search was useful because the known reporting field `+0xA7` also appeared among the bytes that became `0x10` after connection establishment. This confirmed that the diff was capturing connection-related state and could potentially reveal other key-size-related fields.

However, a purely dynamic search is incomplete. An upstream key-size parameter may already be initialized to `0x10` before the connection and remain stable afterward. Such a field would not appear as changed in a pre/post diff. Therefore, we also searched for stable aligned values equal to `0x10`, especially 16-bit and 32-bit aligned values.

8.2 Candidate Filtering

The raw candidate set contained many false positives. We filtered candidates manually before testing them. Addresses were discarded when the surrounding memory suggested that they belonged to:

- pointer-like fields;
- SDP or L2CAP buffers;
- code-like or literal-pool regions;
- structural fields whose modification broke the connection;
- values that were actually part of larger constants such as `0x00001000`.

The remaining candidates were tested by modifying the candidate value to `0x01`, establishing or maintaining a controlled connection, and checking whether InternalBlue and HCI Read Encryption Key Size naturally reported a 1-byte key size without writing directly to `+0xA7`.

8.3 Candidate Testing Results

Table II summarizes the relevant candidates inspected or tested during the upstream search. The table includes both candidates that were actively modified and candidates that were discarded after contextual inspection.

8.4 Interpretation

The candidate search did not reveal any RAM field that behaved as an upstream key-size input. Several fields were writable and persistent, but modifying them did not change the key size reported after connection establishment. Other fields were restored by the firmware, appeared to be unrelated protocol buffers, or caused instability when modified.

The most important outcome is that none of the tested candidates produced the behavior required for a real upstream effect:

candidate = 0x01 → natural key size report of 1 byte
without writing directly to the reporting field at +0xA7.

8.5 Conclusion

The RAM candidate search supports the conclusion that the simple writable-RAM-variable approach is not sufficient to reproduce a real KNOB downgrade on the tested Raspberry Pi 5 / BCM4345C0 platform. The known +0xA7 field can modify the value reported through InternalBlue and HCI, but the tested upstream candidates do not cause the controller to negotiate or naturally report a 1-byte key size.

Therefore, further progress toward a complete KNOB replication would require capabilities beyond static RAM-field modification, such as LMP-level packet modification, firmware patching before key derivation, or external BR/EDR ciphertext capture.

9. RESULTS

This section summarizes the main experimental results obtained during the KNOB replication attempt on the Raspberry Pi 5 / BCM4345C0 platform. The results are organized according to the strength of the evidence: reported downgrade, timing validation, target-side observation, upstream-candidate search, and overall feasibility.

9.1 Summary of Main Findings

Table III summarizes the key observations and their interpretation.

9.2 Reported Downgrade

The initial experiment successfully changed the key size reported by both InternalBlue and the local HCI interface. Writing 0x01 to the field at offset +0xA7 caused InternalBlue to report an effective key length of 1 byte, and HCI Read Encryption Key Size returned the same value.

This result demonstrates that the controller RAM field at +0xA7 influences the local reported key size. However, the result is limited to the reporting layer. It does not establish

that the LMP key-size negotiation selected $N = 1$, nor that the active encryption key was derived with 1 byte of entropy.

9.3 False Positive Confirmed

The decisive validation was the timing analysis. The first key-size read immediately after Encryption Change returned 16 bytes. Altered values such as 0 and 1 appeared only later, after direct RAM writes to the local reporting field.

This timing rules out the interpretation that the later values were used during the original encryption setup. We therefore classify the +0xA7 method as an HCI/InternalBlue reporting false positive.

9.4 Target-Side Observation

The Samsung HCI snoop log provided an independent target-side timeline. It confirmed that the Samsung device observed the connection establishment, link-key handling, and encryption activation. However, the log did not contain a target-side HCI Read Encryption Key Size command.

Therefore, the target-side trace supports the validity of the connection experiment but does not provide direct evidence of the negotiated key size. It cannot confirm either $N = 1$ or $N = 16$.

9.5 Upstream Candidate Search

The search for upstream RAM inputs did not identify a field capable of producing a natural 1-byte key-size result. Several candidates were writable and persistent, but modifying them did not change the final reported key size. Other candidates were restored by firmware, corresponded to unrelated structures, or caused instability.

The key result is that no tested candidate satisfied the required condition:

candidate = 0x01 → key size naturally reported as 1 byte,
without directly modifying the +0xA7 reporting field.

9.6 Feasibility Result

Table IV compares the capabilities required for a complete KNOB replication with the capabilities available in our setup.

With the available setup, we can demonstrate a local HCI/InternalBlue reporting manipulation, but we cannot demonstrate a complete KNOB attack. A complete replication would require evidence that the LMP-negotiated value N was reduced before encryption setup, or equivalent cryptographic evidence from encrypted BR/EDR traffic.

9.7 Final Result

The final result of our analysis is therefore negative with respect to full KNOB replication, but positive with respect to identifying a validation pitfall. On Raspberry Pi 5 / BCM4345C0, the reported key size can be modified in controller RAM so that InternalBlue and HCI report a 1-byte key. However, the timing evidence shows that this value is not sufficient evidence of a real entropy downgrade.

TABLE II: Summary of RAM candidates inspected during the search for an upstream key-size input.

Address / Region	Origin	Action	Observed result	Classification
+0xA7 / 0x20557F	Dynamic candidate; known connection field	Directly written to 0x01	InternalBlue and HCI report 1 byte, but timing validation shows the value changes only post-encryption	Reporting field / false positive
0x2002A8	Dynamic candidate, became 0x10	Attempted write	Write did not persist; value was not controllable through simple writemem	Not persistent
0x2002CC	Dynamic candidate, became 0x10	Attempted write	Write did not persist; value was restored or remained unchanged	Not persistent
0x206484	Dynamic candidate	Written to 0x01	Write persisted, but the connection still reported 16-byte key size	Writable, no effect
0x219190 0x219192	/ Dynamic candidates	Context inspected	Surrounding bytes decoded as a pointer-like field	Discarded
0x21D66E 0x21D675	/ Dynamic candidates	Context inspected	Surrounding memory showed SDP/L2CAP-like protocol data	Discarded
0x20EBC0	Stable 0x0010 field	Characterized with multiple writes	Freely writable, but no evidence that it controls the negotiated key size	Writable, no confirmed effect
0x20EBC2	Stable 0x0010 field near 0x20EBC0	Written to 0x01	Restored to 0x10 during connection establishment; final key size remained 16 bytes	Restored by firmware
0x201FE8	Stable aligned 0x10 candidate	Attempted write	Value did not become the expected 0x01; context appeared code/literal-pool-like	Not controllable / discarded
0x20360C	Stable aligned 0x10 candidate	Written to 0x01	Write persisted, but key size remained 16 bytes	Writable, no effect
0x208A64	Stable candidate in structured region	Written to small values	Connection failed or controller became unstable	Unsafe structural field
0x20A135, 0x20A14D, 0x20A165, 0x20A17D	Stable byte-level hits	Context inspected	Values were part of larger 0x00001000 constants, not isolated key-size fields	Discarded
0x200704	Stable aligned 0x10 candidate	Written to 0x01	Write persisted, but key size remained 16 bytes	Writable, no effect
0x201858	Stable aligned candidate	Tested	No useful effect on the reported key size	No effect
0x203660	Clean stable aligned 0x10 candidate	Written to 0x01	Connection succeeded, but InternalBlue and HCI still reported 16 bytes	Tested, no effect
0x20F75C	Clean stable aligned 0x10 candidate	Written to 0x01	Connection succeeded, but key size remained 16 bytes	Tested, no effect
0x210D1E	Isolated stable 0x0010 candidate	Written to 0x01	Connection succeeded, but key size remained 16 bytes	Tested, no effect
0x210D1C	Isolated stable 0x0010 candidate	Written to 0x01	Connection succeeded, but key size remained 16 bytes	Tested, no effect

The correct conclusion is:

The +0xA7 method produces an HCI/InternalBlue reporting false positive, not a complete KNOB replication.

10. DISCUSSION

The experiments show that the Raspberry Pi 5 / BCM4345C0 setup can produce an apparent KNOB-like downgrade at the HCI and InternalBlue level, but cannot, with the available capabilities, demonstrate a complete LMP-level entropy downgrade. This distinction is the central result of the work.

10.1 Meaning of the False Positive

The most important finding is that the field at offset +0xA7 controls the key size reported by InternalBlue and by HCI Read Encryption Key Size, but it is not sufficient evidence of the value negotiated during LMP key-size ne-

gotiation. The timing trace shows that immediately after Encryption Change the controller still reports a 16-byte key size. Values such as 0 and 1 appear only later, after direct RAM writes.

This demonstrates that the local reported key size can be modified after encryption has already been enabled. Therefore, the apparent downgrade is a reporting artifact, not proof that the encryption key used by the BR/EDR link was derived with 1 byte of entropy.

This result is significant because it exposes a validation pitfall: observing Effective Key Len = 1 in InternalBlue, even when confirmed by the HCI command Read Encryption Key Size, is not enough to claim a successful KNOB attack when the attacker can modify local controller RAM.

TABLE III: Summary of the main experimental findings.

Experiment	Observation	Interpretation
Direct write to +0xA7	InternalBlue reports Effective Key Len = 1 byte	The field controls the key size reported by InternalBlue, but this alone does not prove a real LMP-level downgrade.
HCI Read Encryption Key Size after +0xA7 write	HCI also reports key size 0x01	The local HCI response reads a controller-maintained state field affected by the RAM write.
Timing validation	Immediately after Encryption Change, the controller reports key size 16	The encrypted link is established with the normal reported key size before post-encryption modifications occur.
Post-encryption writes	Later reads report altered values such as 0 and 1	HCI/InternalBlue key-size reporting is mutable after encryption and can produce false positives.
Traffic continuity	L2CAP traffic continues after post-encryption modification of the reported key-size field	The RAM write does not appear to replace the live cryptographic state of the active link.
Samsung-side HCI snoop	The target log shows connection and encryption activation, but no target-side HCI Read Encryption Key Size	The Samsung log confirms encryption occurred, but is inconclusive for the negotiated entropy N .
Upstream RAM candidates	No tested candidate caused a natural 1-byte key-size report without writing +0xA7	No upstream key-size input was identified through the tested writable-RAM approach.

TABLE IV: Capabilities required for a complete KNOB replication.

Capability	Available	Outcome
Local RAM read/write	Yes	Enables reporting-field manipulation
HCI tracing	Yes	Shows host-controller state
Target-side HCI snoop	Partial	Confirms encryption, not N
LMP packet visibility	No	Cannot directly observe negotiation
LMP packet modification	No	Cannot force key-size negotiation
ROM / patchram hook	No	Cannot patch key derivation path
BR/EDR ciphertext capture	No	Cannot prove brute-forceability

10.2 Why HCI-Level Validation Is Insufficient

HCI is the interface between the host and the controller. It can expose controller-maintained state, but it does not necessarily reveal the LMP messages exchanged between controllers. In our setup, HCI Read Encryption Key Size reports a value stored or maintained by the local controller. Since that value can be modified through RAM writes, it cannot be treated as independent evidence of the LMP-negotiated entropy.

A rigorous KNOB validation requires a stronger form of evidence. At least one of the following would be needed:

- direct observation of the LMP key-size negotiation showing $N = 1$;
- modification of the LMP negotiation before encryption setup;
- firmware-level evidence that the reduced entropy is used before key derivation;
- cryptographic evidence from encrypted BR/EDR traffic showing that the link can be brute-forced with a 1-byte key space.

Our setup provides none of these capabilities directly. It provides local RAM access and HCI-level observation, which

are useful for firmware analysis but insufficient for proving the actual negotiated entropy.

10.3 Comparison with the Original KNOB Attack

The original KNOB attack acts at the level of the Bluetooth key-size negotiation. The adversary must influence the unauthenticated LMP negotiation before the final encryption key is derived. In contrast, our Raspberry Pi 5 experiments modify a local controller RAM field after the connection is already established.

This difference explains why the apparent downgrade cannot be considered equivalent to the original attack. The original attack changes the protocol decision that determines N ; our initial method changes a field that affects the value later reported to the host. These are different levels of the Bluetooth stack.

Consequently, reproducing the original attack on this platform would require capabilities that go beyond the current setup, such as an LMP hook, reliable firmware patching before encryption setup, or external ciphertext capture.

10.4 Implications for Raspberry Pi 5 / BCM4345C0

The results suggest that the Raspberry Pi 5 / BCM4345C0 platform is useful for local controller-state analysis but limited for complete KNOB replication. InternalBlue can access and modify selected RAM fields, and this allowed us to map parts of the connection structure and identify the reported key-size field. However, the platform did not provide reliable access to the lower-level mechanisms required to alter or verify LMP key-size negotiation.

The upstream candidate search further supports this conclusion. None of the tested RAM candidates caused a natural 1-byte key-size report without directly modifying +0xA7. This suggests that the true source of the negotiated key-size value is not exposed as a simple writable RAM field in the tested regions. It may instead be a transient variable, a firmware-internal structure, an LMP packet buffer, or a value controlled

by code paths not reachable through the available RAM-write approach.

10.5 Value of the Negative Result

Although the experiments do not demonstrate a complete KNOB attack, the negative result is still useful. The work identifies a concrete false-positive pattern that could otherwise lead to an incorrect claim of attack success:

```
InternalBlue key size = 1 and HCI key size  
                        = 1
```

do not imply:

```
LMP negotiated  $N = 1$ .
```

This distinction is especially important in Bluetooth firmware experimentation, where local controller memory can be modified and debugging interfaces may expose mutable state rather than protocol-level ground truth.

The main contribution of the work is therefore not a full KNOB replication, but a feasibility assessment of the Raspberry Pi 5 / BCM4345C0 platform and a demonstration that HCI/InternalBlue key-size reporting can be misleading when used as the only validation criterion.

11. LIMITATIONS

This section summarizes the main limitations of our experimental setup. These limitations define the scope of our conclusions and explain why the observed HCI/InternalBlue downgrade cannot be considered a complete KNOB replication.

11.1 No Direct LMP Visibility

The main limitation is the lack of direct visibility into the LMP key-size negotiation. The KNOB attack acts on LMP messages exchanged between Bluetooth controllers, while our main traces expose HCI-level activity between the host and the local controller. Although `btmon` and Samsung `btsnoop` logs were useful to reconstruct the connection timeline, they did not provide direct evidence of the LMP-negotiated value N .

As a result, we could not directly observe whether the controllers exchanged or accepted a key-size value of 1 byte.

11.2 No Reliable LMP Modification

A complete KNOB replication would require modifying the key-size negotiation before encryption setup. In our setup, we did not have a reliable hook to intercept or modify LMP key-size negotiation packets before the final encryption key was derived.

This limitation prevents us from forcing the actual LMP negotiation to select $N = 1$ in the same way as the original KNOB attack.

11.3 No Reliable ROM or Patchram Access

The Raspberry Pi 5 / BCM4345C0 platform allowed partial RAM inspection and modification, but did not provide reliable

access to the relevant ROM code or to a stable patchram mechanism for modifying the firmware path responsible for key-size negotiation.

Therefore, we could not patch the controller firmware at the point where LMP key-size requests are generated, parsed, or applied.

11.4 No Over-the-Air Ciphertext Capture

Our setup did not include an external BR/EDR sniffer capable of capturing encrypted over-the-air traffic. This is important because a cryptographic validation of KNOB would require showing that the encrypted traffic can be brute-forced with a 1-byte entropy key space.

The local HCI traces are insufficient for this purpose because traffic delivered to the host may already have been processed by the controller.

11.5 Target-Side HCI Logs Are Inconclusive for Key Size

The Samsung HCI snoop log confirmed connection establishment and encryption activation, but it did not contain a target-side HCI Read Encryption Key Size command. Therefore, the target-side trace was useful for validating the connection timeline, but it did not reveal the key size perceived by the Samsung controller.

11.6 Scope of the Conclusion

Our conclusion is not that KNOB is impossible on every Raspberry Pi 5 or on every BCM4345C0-based configuration. Rather, our conclusion is limited to the capabilities available in our experimental setup.

With local RAM read/write, HCI tracing, InternalBlue, and target-side HCI logs, we can demonstrate that the reported key size can be manipulated. However, we cannot demonstrate that the actual LMP-negotiated entropy was reduced.

Therefore, our final claim is deliberately scoped as follows:

With the available Raspberry Pi 5 / BCM4345C0 setup, we can demonstrate an HCI/InternalBlue reporting false positive, but not a complete KNOB replication.

12. CONCLUSION

This work investigated the feasibility of reproducing a KNOB-style Bluetooth BR/EDR entropy downgrade on a Raspberry Pi 5 equipped with a BCM4345C0/CYW43455 controller. The initial experiments appeared to indicate a successful downgrade, since modifying the connection-structure field at offset `+0xA7` caused both InternalBlue and HCI Read Encryption Key Size to report a 1-byte key size.

A deeper validation showed that this interpretation was incorrect. The decisive timing evidence showed that the first key-size reading immediately after Encryption Change still reported 16 bytes, while altered values such as 0 and 1 appeared only later, after direct RAM writes. We therefore classify the `+0xA7` method as an HCI/InternalBlue reporting false positive rather than a complete KNOB replication.

We also searched for upstream RAM fields that could influence the key-size value before or during LMP negotiation. This search included both dynamic candidates identified through pre/post-connection RAM diffs and stable aligned 0×10 candidates. None of the tested candidates caused a natural 1-byte key-size report without directly modifying the $+0 \times A7$ reporting field.

The final result is therefore a scoped negative replication. With the available Raspberry Pi 5 / BCM4345C0 setup, we can manipulate the key size reported to the host, but we cannot demonstrate that the actual LMP-negotiated entropy was reduced. A complete KNOB replication would require additional capabilities, such as LMP-level visibility or modification, reliable firmware patching before key derivation, or external BR/EDR ciphertext capture.

The main contribution of this work is the identification and validation of a realistic experimental pitfall: HCI/InternalBlue key-size reporting can be misleading when local controller RAM is writable. As a result, future KNOB replication attempts on similar platforms should not treat a reported key size of 1 byte as sufficient evidence of a real cryptographic downgrade unless it is supported by LMP-level or ciphertext-

level validation.

AI ASSISTANCE DISCLOSURE

AI-based tools were used during the project as support for code drafting, command-line troubleshooting, log interpretation, and language refinement. All experiments, controller interactions, collected logs, technical decisions, and final conclusions were performed, verified, and critically reviewed by the authors.

REFERENCES

- [1] D. Antonioli, N. O. Tippenhauer, and K. B. Rasmussen, “The knob is broken: Exploiting low entropy in the encryption key negotiation of bluetooth br/edr,” in *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, 2019.
- [2] Bluetooth SIG, “Bluetooth core specification: Br/edr controller, link manager protocol specification,” <https://www.bluetooth.com/wp-content/uploads/Files/Specification/HTML/Core-54/out/en/br-edr-controller/link-manager-protocol-specification.html>, accessed: 2026-05.
- [3] —, “Bluetooth core specification: Br/edr controller, security specification,” <https://www.bluetooth.com/wp-content/uploads/Files/Specification/HTML/Core-54/out/en/br-edr-controller/security-specification.html>, accessed: 2026-05.